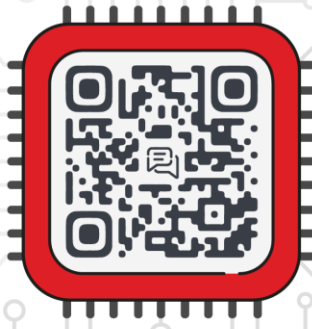




نظري

6

10

1500 sp

كلية الهندسة المعلوماتية

السنة الرابعة - قسم البرمجيات

OO Testing & Web App Testing

د. سعيد أبو طراب



RB Informatcs; 13/05/2026

هندسة برمجيات ٢

في المحاضرة السابقة ركّزنا على فهم كيفية اختبار البرنامج من الداخل عبر المسارات والتفرعات وتدفق البيانات. والآن نتنقل خطوة أبعد، إلى كيف تتغير فلسفة الاختبار عندما يصبح النظام كائني أو يعمل عبر الويب.

OO Testing

إنّ النظام لم يعد مجرد مجموعة دوال منفصلة، بل أصبح مجموعة كائنات تتفاعل مع بعضها البعض. وهنا تظهر أول مشكلة إذا كان لدي كائن (Object) يعتمد في عمله على كائنات أخرى، فهل يكفي أن أختبر كل دالة (Function) لوحدها؟ لا

وهنا نبدأ أنواع الاختبارات الخاصة ب OO:

1. Unit Testing

عندما نقول: Unit Testing فما هي الوحدة (Unit) التي سأختبرها؟ في البرامج التقليدية كان الجواب غالباً: الدالة (Function)

أما هنا فالجواب أصبح الصف (Class) لأن الكلاس لا يتكون فقط من دوال بل يحتوي:

State	Attributes
Relationships	Methods

وجميعها تعمل معاً .

تخلوا أن لدينا الكلاس التالي:

```
class BankAccount {
    double balance;
    deposit();
    withdraw();
    getBalance();
}
```



لو أردنا اختبار الدالة withdraw():
فهل يكفي أن أستخدمها وأرى النتيجة؟

ليس دائماً، لأن هذه الدالة تعتمد على balance أي على حالة الكائن. وهنا تظهر فكرة مهمة جداً

الدوال داخل الكلاس ليست مستقلة تماماً بل تؤثر ببعضها البعض.
فعندما تنفذ:

deposit() فإنها تغير حالة الكائن. وعندما تنفذ بعدها withdraw() فإن نتيجة التنفيذ أصبحت مختلفة.

إذاً لا يمكنني النظر لكل دالة بمعزل عن بقية الكلاس.

مثال:

افترض أن الرصيد الحالي: $Balance = 100$

وقمنا بتنفيذ: $Withdraw(50)$

سيصبح: $Balance = 50$

لكن إذا بدأنا من: $Balance = 0$

ونفذنا العملية نفسها: $Withdraw(50)$

فقد نحصل على:

أو رصيد سالب

رفض العملية

رسالة خطأ

بحسب تصميم النظام لاحظوا أن الدالة نفسها لم تتغير، الذي تغير هو حالة الكائن.

ملاحظة مهمة: في الأنظمة الكائنية نهتم كثيراً بما يسمى: حالة الكائن. لأن كثيراً من الأخطاء تظهر نتيجة الانتقال من حالة إلى حالة أخرى.

2. Integration Testing

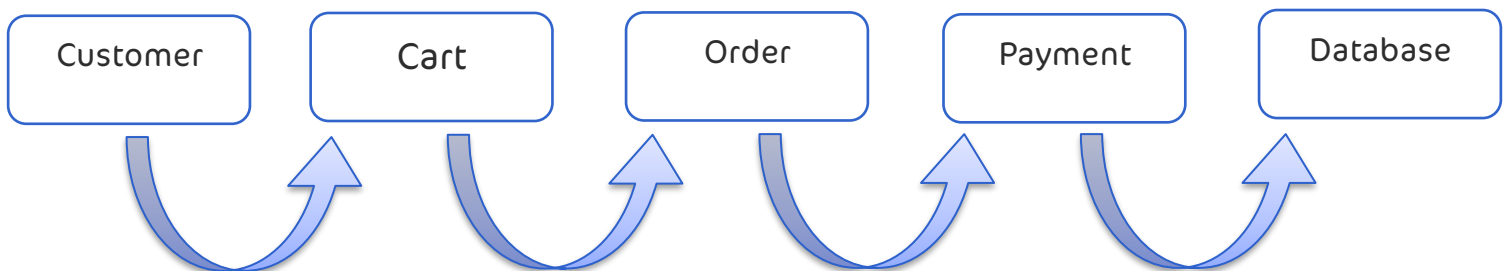
بعد الانتهاء من اختبار كل Class على حدة، نصل إلى سؤال هل أصبح النظام جاهزاً؟
الإجابة: لا، لأن معظم الأخطاء الحقيقية لا تظهر داخل الكلاس نفسه بل تظهر عندما تتفاعل الكلاسات مع بعضها البعض.
هذا يقودنا إلى مستوى أعمق من الاختبار يتعلق بتفاعل الكلاسات مع بعضها.

1. Thread-Based Testing

لذلك دعونا نفكر في حدث واحد فقط.

ضغط المستخدم على زر "شراء" ماذا يحدث بعدها؟

قد يتم تنفيذ التسلسل التالي:



كل هذه الخطوات بدأت من حدث واحد فقط

إذا فشلت أي خطوة من هذه الخطوات فالمسار كله يفشل.

نحن لا نختبر كلاساً منفرداً، بل نختبر رحلة تنفيذ كاملة.

2. Use-Based Testing

دعونا ننظر إلى الموضوع بطريقة مختلفة، بدل أن نطلق من الكود سنطلق من المستخدم.
نسأل: ماذا يريد المستخدم أن يفعل؟ وهنا يظهر مفهوم: حالة الاستخدام (Use Case)
نفترض أن لدينا:

Use Case بعنوان: "تسجيل الدخول" هذه العملية تتضمن:

- إدخال اسم المستخدم
- إدخال كلمة المرور
- التحقق من قاعدة البيانات
- إنشاء جلسة المستخدم

بدل أن أختبر:

ValidatePassword() وحدها، أختبر العملية كاملة Login Scenario : من البداية للنهاية

3. Cluster Testing

لنفترض أن لدينا مجموعة من الكلاسات تعمل دائماً مع بعضها.
مثل:

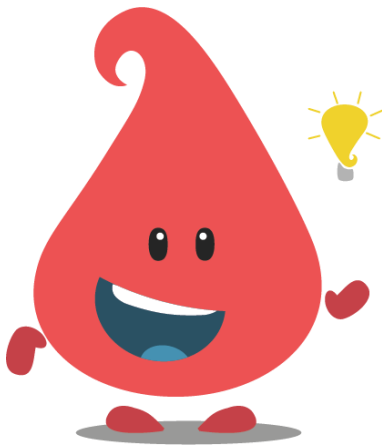
Payment

Order

Customer

في الواقع يصعب فصلها. فماذا نفعل؟

نجمعها معاً ونختبرها كوحدة واحدة وهذا يسمى: Cluster Testing

مثال:

في متجر إلكتروني عندما ينشئ العميل طلباً فإن:
Customer يتفاعل مع Order ثم مع Payment بشكل دائم
لذلك من المنطقي اختبار هذه المجموعة معاً.

خطأ شائع

أن يتم اختبار كل Class بصورة ممتازة لكن لا يتم اختبار التعاون بينها. ثم يتفاجأ الفريق بظهور الأخطاء بعد دمج النظام.

3. Fault-Based Testing

حتى هذه اللحظة كنا نبني الاختبارات بهدف تغطية النظام، لكن ماذا لو بدأنا من الاتجاه المعاكس حاولنا التنبؤ بالأخطاء قبل حدوثها؟

هنا يظهر: الاختبار المعتمد على الأخطاء. (Fault-Based Testing)

الفكرة بسيطة نسأل أنفسنا لو كنت مبرمجاً وارتكبت خطأ، فأين يمكن أن يكون هذا الخطأ؟ ثم نبني اختباراً مخصصاً لاكتشافه. أي أننا لا نختبر عشوائياً بل نبحث عن أخطاء متوقعة.

مثال:

لدينا نظام بنكي المبرمج ربما نسي التحقق من الرصيد قبل السحب إذا نصمم الحالة التالية:

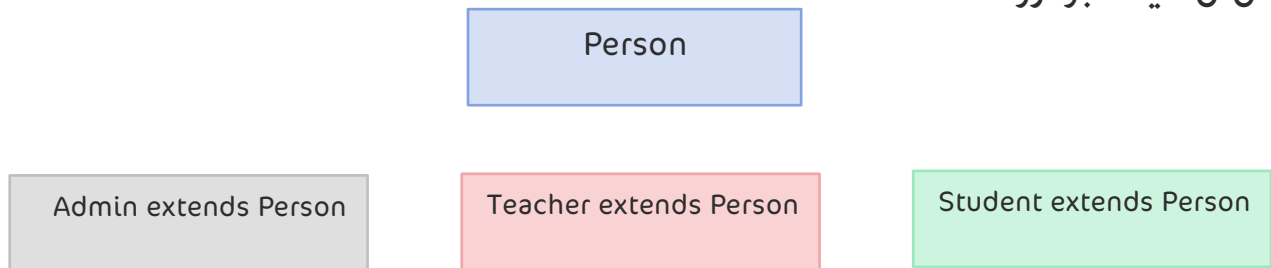
Withdraw = 500

Balance = 100

ثم نراقب النتيجة إذا سمح النظام بالسحب فلدينا خطأ.

4. Class Testing & Class Hierarchy Testing

أحد أهم مميزات البرمجة الكائنية وهي: الوراثة (Inheritance) لنفترض أن لدينا شجرة وراثة كاملة:



السؤال إذا اخترنا الكلاس الأب Person هل يكفي ذلك؟

لا، لأن كل كلاس ابن قد يضيف وظائف جديدة، وقد يعدل وظائف موجودة أصلاً، وقد يعيد تعريف بعض السلوكيات.

مثال:

افترض أن: Person يحتوي على login() ، لكن Admin أضاف صلاحيات جديدة أو عدّل طريقة تسجيل الدخول ، هنا قد يظهر خطأ داخل: Admin حتى لو كان Person سليماً تماماً.

5. Partition Testing

دعونا نفكر بالمشكلة التالية: إذا كان لدي ألف قيمة إدخال هل سأختبر الألف كلها؟
طبعاً لا، لأن الوقت لا يكفي ولهذا نحتاج طريقة تقلل عدد حالات الاختبار:
بحيث أقوم بتقسيم المدخلات إلى مجموعات ثم أختار قيمة ممثلة عن كل مجموعة.



Success is a journey
not a destination

إذا كانت المجموعة كلها تتصرف بالطريقة نفسها فإن اختبار عنصر واحد منها يكفي غالباً.

1. State-Based Partitioning

في هذا النوع نعتمد على حالة الكائن.

لدينا حساب بنكي قد يكون في إحدى الحالات التالية:

Active ▪ | Frozen ▪ | Closed ▪ |

كل حالة تمثل سلوكاً مختلفاً، إذا أردت اختبار السحب: فلا يكفي أن أختبر حالة واحدة فقط.

لأن: Withdraw(100)

قد ينجح عندما تكون الحالة: Active

وقد يفشل عندما تكون: Frozen

2. Attribute-Based Partitioning

في هذا النوع نعتمد على خصائص الكلاس.

مثال

لدينا موظف له المعلومات التالية:

name , age , salary , department

يمكن تقسيم الاختبارات حسب القيم المختلفة لهذه الخصائص.

مثلاً: العمر:

▪ أقل من 18

▪ 18-60

▪ أكثر من 60

كل مجموعة تعتبر Partition مستقلة.

3. Category-Based Partitioning

في هذا النوع نقسم الاختبارات حسب نوع العملية نفسها.

مثال

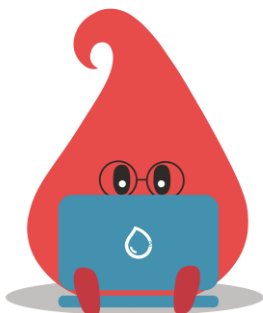
في نظام إدارة طلاب لدينا:

▪ Add Student

▪ Delete Student

▪ Update Student

▪ Search Student

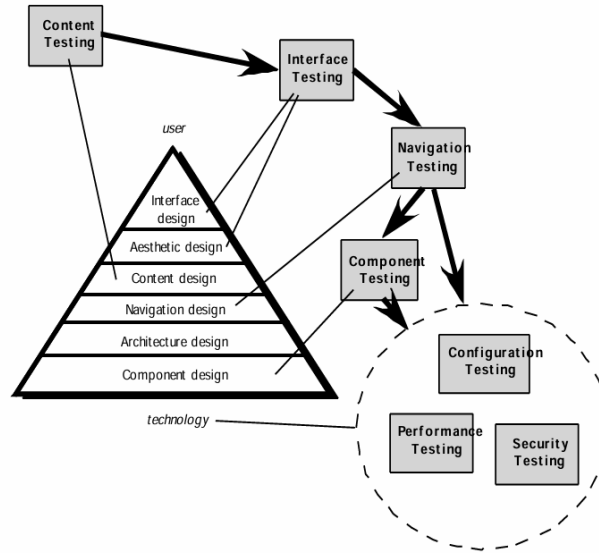


كل فئة تمثل Category مختلفة، وبالتالي تحتاج اختبارات مستقلة

ملاحظة مهمة:

الهدف من التقسيم ليس زيادة الاختبارات، بل تقليلها مع الحفاظ على مستوى جيد من التغطية.

Web Testing



النوع الآخر من الأنظمة هو: تطبيقات الويب (Web Applications)

عندما أقول إن الموقع يعمل، ماذا أقصد بالضبط؟

هل أقصد:

أن الصفحات تظهر؟ أن الروابط تعمل؟ أن الأداء جيد؟ أن المحتوى صحيح؟ أن النظام آمن؟
الموقع قد يعمل وظيفياً بصورة ممتازة.

لكن:

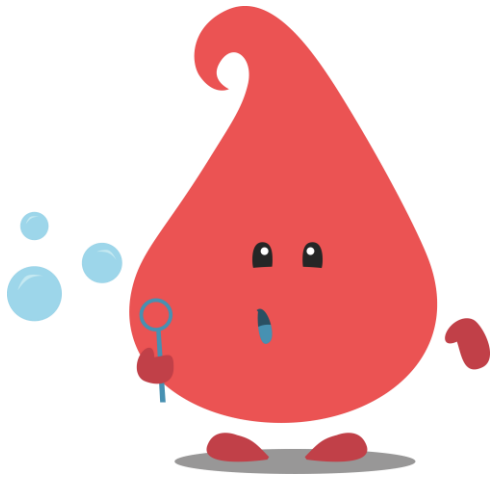
- بطيء جداً
- صعب الاستخدام
- غير آمن.
- يحتوي روابط

وفي جميع هذه الحالات سنقول إن الموقع ما زال يحتوي مشاكل.

لو دخل مستخدم إلى موقعك لأول مرة، كيف سيحكم عليه؟

هل سيذهب لقراءة الكود؟

طبعاً لا، سيحكم عليه من خلال: أبعاد جودة الاختبار (Testing quality dimensions):



- المحتوى (Content)
- الوظيفة. (Function) .
- البنية (Structure)
- قابلية الاستخدام (Usability)
- قابلية التنقل (Navigability)
- الأداء (Performance)
- التوافق (Compatibility)
- التشغيل (Interoperability)
- الأمان (Security)

ولهذا فإن اختبار مواقع الويب لا يقتصر على الوظائف البرمجية فقط، بل يشمل جوانب كثيرة أخرى:

1. Content Testing

البعض يعتقد أن مهمة المختبر تنتهي عندما يتأكد أن الأزرار تعمل، فلو أن الموقع يعمل بشكل مثالي لكن توجد معلومة خاطئة فهل الموقع صحيح؟
الإجابة لا، لأن المستخدم جاء للحصول على معلومة صحيحة وليس فقط لرؤية صفحة تعمل.

ما الذي نختبره؟

نراجع:

- | | | |
|-------------------|--------------------|---------------------|
| ■ صحة المعلومات | ■ اكتمال المعلومات | ■ الأخطاء الإملائية |
| ■ الأخطاء اللغوية | ■ حداثة المعلومات | ■ تناسق المحتوى |

2. User Interface Testing

وفق هذا الاختبار نقوم بما يلي:

1. يتم اختبار خصائص الواجهة لضمان تقديم المحتوى البصري والجمالي (Aesthetics) وفقاً لقواعد التصميم المعتمدة (Design Rules) وبشكل خال تماماً من العيوب.
2. تختبر الواجهة الفردية بشكل منفصل بأسلوب يحاكي اختبارات الوحدة البرمجية. (Unit Testing) أي ال unit test هنا تعتبر الواجهة.
3. يخضع كل عنصر تفاعلي للاختبار ضمن سياق حالة استخدام محددة (Use-Case) تستهدف شريحة معينة من فئات المستخدمين (User Category) .
4. يتم فحص الواجهة بكامل مكوناتها بناءً على سيناريوهات استخدام منتقاة لرصد أي خلل في منطق ودلالات التعامل مع النظام (Interface Semantics) .

5. تجرى اختبارات الواجهة ضمن بيئات عمل ومتصفحات متنوعة (Browsers) للتأكد من توافقها (Compatibility) وصحة عرضها.

3. Navigability Testing

هل يستطيع المستخدم الانتقال بين صفحات الموقع بسهولة؟

1. الروابط (Links)

إذا ضغط المستخدم على رابط فيجب أن يصل إلى الصفحة الصحيحة.
أما إذا حصل على: 404 Not Found فهذا لدينا مشكلة تنقل.

2. Redirects

أحياناً يتم حذف صفحة أو تغيير عنوانها فماذا يحدث؟
يجب تحويل المستخدم إلى المكان الجديد وهذا يسمى: **إعادة التوجيه (Redirect)**

3. Site Map

خريطة الموقع (Site Map) ، هل جميع الروابط الموجودة فيها صحيحة؟

4. Search

البحث داخل الموقع
مثال: إذا كتبت:

Software Testing وكان المقال موجوداً لكن البحث لم يعثر عليه فهذه مشكلة في التنقل والوصول للمعلومات.

5. Frames and Framesets

تختبر للتأكد من صحة المحتوى، سلامة التخطيط، الحجم، أداء التنزيل، والتوافق مع المتصفحات.

6. Bookmarks

يجب اختيار صفحة ذي معنى عند إنشاء الإشارة المرجعية.

4. Security Testing

لو أنشأت موقعاً:

▪ سريعاً جداً | ▪ جميل التصميم | ▪ جميع الوظائف تعمل

لكن أي شخص يستطيع سرقة البيانات منه هل سنعتبره ناجحاً؟ طبعاً لا
بل قد يكون فاشلاً بالكامل مهما كانت ميزاته الأخرى ولهذا يعتبر الأمان أحد أهم معايير جودة النظام.

ما الذي نختبره؟

عادة يتم التركيز على ثلاثة أماكن رئيسية:

1. العميل (Client)

- المتصفح
- الواجهة الأمامية

2. الشبكة (Network)

البيانات أثناء انتقالها بين المستخدم والخادم.

3. الخادم (Server)

وهو المكان الذي يحتوي:

- قواعد البيانات
- منطق النظام
- الحسابات

5. Performance Testing

نفترض أن موقعك يعمل بصورة صحيحة لكن كل عملية تحتاج 15 ثانية لتنفيذها هل سيكون المستخدم سعيداً؟ غالباً لا، لذلك فإن السؤال هنا ليس هل يعمل النظام؟ بل ما مدى كفاءة عمله؟

نريد معرفة:

- سرعة الاستجابة
- عدد المستخدمين الذين يستطيع النظام خدمتهم
- متى يبدأ الأداء بالتراجع
- أين تظهر نقاط الاختناق

مثال:

لدينا متجر إلكتروني: في الصباح يوجد 50 مستخدم وكل شيء ممتاز.

لكن أثناء التخفيضات أصبح لدينا 5000 مستخدم في الوقت نفسه.

هنا تبدأ الأسئلة الحقيقية: هل سيبقى النظام سريعاً؟

هل ستستمر قاعدة البيانات بالاستجابة؟

هل سيتحمل الخادم هذا العدد؟

6. Load Testing

7. Stress Testing

ما الفرق بين اختبار الحمل (Load Testing) واختبار الإجهاد (Stress Testing) ؟

■ اختبار الحمل (Load Testing)

في النظام البرمجي لدينا معاملات أو عمليات (Transactions) أي تفاعلات تحدث مع التطبيق. إذا كان لدينا ثلاثة مستخدمين ينفذون عمليات على النظام، فإنهم يولدون حملاً (Load) على النظام. ويحسب الحمل وفق عدة عوامل:

عدد المستخدمين × عدد العمليات × حجم البيانات

Load = Number of Users × Number of Transactions × Size of Data

$P = N * T * D$

■ اختبار الإجهاد (Stress Testing)

الهدف هنا هو معرفة: ما الحد الذي يستطيع النظام تحمله قبل أن يفشل؟



ما زرع الله في قلبك رغبة الوصول لأمر معين، إلا لأنه يعلم أنك ستصل إليه

حتى هذه اللحظة ركزنا على جانب مهم جداً من هندسة البرمجيات، وهو التأكد من جودة المنتج البرمجي من خلال عمليات الاختبار المختلفة.

وبهذا نكون قد طوينا صفحة الاختبارات (Testing)

لنبدأ برحلة في عالم Umbrella Activities

